1.1 ANALYSIS OF ALGORITHMS

Let us consider the algorithm of estimating factorial given in example 1.7 to start a discussion on *how to analyse an algorithm*. The algorithm of estimating factorial is shown below.

Name: Factorial
Input: Integer number n
Output: Factorial of n
Description: $n! = n \times (n-1) \times (n-2) \times \ldots \times 3 \times 2 \times 1$.

BEGIN		
1.	*Initialize fact = 1*	Executed 1 time
2.	*Repeat till n >= 1*	Executed n+1 times
3.	*fact = fact * n*	Executed n times
4.	*n = n - 1*	Executed n times
5.	*End Repeat*	
6.	*Print fact*	Executed 1 time
END		

To simplify the analysis, let us assume that each of the basic computation in the algorithm take constant amount of time, *say c*. Now, let us ask a basic question on *how many times, the basic operations will be computed?* Without difficulty, it can be seen that the initialization statement *fact = 1* in Step-1 will be executed only one time. The comparison statement $n >= 1$ in Step-2 will be executed $n + 1$ times. The statement $fact = fact * n$ in Step-3 will be executed n times. The statement $n = n - 1$ in Step-4 will be executed n times. Step-6 will be executed only one time. The total time for execution is $T(n) = 1c + (n+1)c + nc + nc + 1c = 3nc + 3c$. If c is considered to be unit time, it can be further simplified as $T(n) = 3n + 3$, which is a polynomial of degree one. It basically means, the running time of the algorithm linearly increases with the increase in the value of the input. Further, if the expression $T(n) = 3n + 3$ is looked at deeper, the second term 3 also means that there are three statements which are independent of the input value, and the coefficient 3 of independent variable n also means that there are three statements which will be repeated n times. Depending on the value of n, the running time will also be different, i.e., linearly increases with the increase in n.

What about the space consumption? Space complexity of an algorithm will be defined not only by the storage requirement of the data that the algorithm needs during its computation, but also by the process of the program of the algorithm during its execution. To understand memory consumption of a program, let us briefly review memory allocation of a process during its execution. When a process is created by the operating system, a chunk of memory is allocated to the process. This chunk is broadly divided into four sections as shown in Figure 1.6 – *text/code, data, heap,* and *stack*. The text/code section stores the executable code of the program that the process will run. The data section stores the global variables defined in the program. The heap will store all the dynamically allocated memory during the execution of the program. The stack section stores the local variables of all the active functional calls. While the code and data sections are static, the stack and heap may grow. When the process performs any dynamic memory allocation (for example, using *malloc* or *calloc* in C programming), it will be added to heap. When a function is called by a process, a record (known as *activation record*) containing all the local variables (including function parameters) defined within the scope of the function, return address to the caller function will be pushed in the stack. The top of the stack will have the information about the currently active function. When the process completes execution of a function, the corresponding records will be popped from the stack. For example, when a C program starts executing its *main()* function, its activation record containing information about its local variables, return address etc. will be pushed into the stack. Likewise, when main function or any function calls another function, the activation record of the newly called function will be pushed into the stack. When process exits the function, its record will be popped from the stack. Thus, the stack at a particular instance will have the records of all the functions whose executions are yet to be completed. The stack and heap will grow in the opposite direction as shown in Figure 1.6. When they touch, either a stack overflow or memory allocation error will occur. Space complexity defines the

S Ranbir Singh - Lecture

rate of growth of the memory consumed by the process in execution (in the data section, heap section and stack section) with respect to the input n.

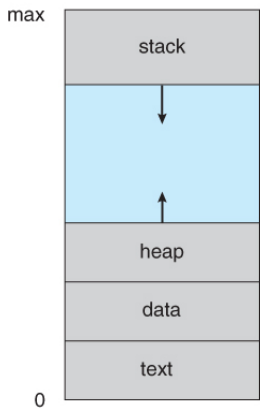


Figure 1: A process in memory

Let us estimate space complexity of the above factorial algorithm/program. As described above, the text section will consume a fixed amount of memory, *say c*. As there is no global variables or dynamic memory allocations, the data and heap section consume zero memory. Only one record, *say r*, for *main()* function will be pushed into stack. Therefore, the algorithm/program will consume a constant size space of $S(n) = c + r$, which does not grow with n.

Now, let us change the above algorithm for calculating the factorial of n from iterative $n! = n\times(n-1)\times(n-2)\times \ldots \times 3\times 2\times 1$ to a recursive form as

$$fact(n) = \begin{cases} n\times fact(n-1), & if\ n > 1 \\ 1, & if\ n = 1 \end{cases}$$

The recursive algorithm and its respective recursive program in C is given below.

```
//ALGORITHM
Name: Factorial (n)
Input: Integer number n
Output: Factorial of n
Description: fact(n) = n×fact(n − 1)
BEGIN
1.  IF n == 1 THEN
2.      RETURN 1
3.  RETURN n * Factorial(n-1)
END
```

```
//PROGRAM
#include <stdio.h>
int fact(int);
main(){
   int n = 10;
   printf("The factorial is %d", fact(n));
}
int fact(int n){
    if(n == 1)
     return 1;
    return n*fact(n-1);
}
```

For the above recursive algorithm, its running time can be defined using the recursive expression $T(n) = T(n-1) + c$ and $T(1) = c$, where c is the time for executing one operation. If $n = 1$, it just returns 1. i.e., it involves one comparison operation i.e., $T(1) = c$. If $n > 1$, then time for one comparison operation (i.e., c) plus time for one multiplication operation (i.e., c) plus time for

Facorial(n-1) (i.e., $T(n-1)$). Therefore, $T(n) = T(n-1) + c + c$. Let us now, simplify the recursive expression and see what we get.

$$T(n) = T(n-1) + c + c = T(n-2) + 2c + 2c = T(n-3) + 3c + 3c = \ ...$$
$$= T(1) + (n-1)c + (n-1)c = c + (n-1)c + (n-1)c = 2nc - c$$

Since c is constant, execution time $T(n) = 2n - 1$ is a polynomial of degree one. It means that the execution time grows linearly with n, which is similar to the above iterative algorithm.

Now, let us estimate its space consumption. Like in iterative algorithm, except for the stack section, the code, data and heap sections of the process will consume fixed memory, i.e., c + 0 + 0, c for code, zeros for heap and data. However, unlike its iterative algorithm, the stack will grow as the process executes the recursive function. As the process starts execution, one record for main function will be pushed into the stack. When main function calls the recursive function for the first time, i.e., *fact(n)*, a record for the fact function will be pushed into the stack. When the fact function calls itself for *fact(n-1)*, another record of fact function will be pushed into the stack. Likewise, one record each will be pushed for every recursive call into the stack, and the stack grows till *fact(1)* is called. At the time the process executes *fact(1)*, the stack has $n + 1$ records. As the recursive function terminates in the reverse order, corresponding records will also be popped from the stack in reverse order. So, the space consumption will be $S(n) = (n+1)r + c$, where r is the fixed memory for each record. For simplicity, if we assume all the constant parameters as one, $S(n) = n + 2$, a polynomial of degree one. Unlike, the iterative algorithm, the space consumption of the recursive algorithm grows linearly with input n.

1.1.1 Running Time and Space consumption of the Example 1.3 Program

The example program 1.3 convert an integer number to its binary equivalent. The corresponding algorithm is also given below. The variable q is initialized to n, and in every iteration, the value of q is reduced by half i.e., $, \frac{n}{2^1}, \frac{n}{2^2}, \frac{n}{3}, \dots, \frac{n}{2^k}$. The subsequent values of q is divided by 2 for k times and k is equal to $\lceil \log_2 n \rceil$. The running time of this algorithm is equivalent to $T(n) = 5\lceil \log_2 n \rceil + 2$. Therefore, running time grows logarithmically with n. In this program, the size (number of elements) of the local variable *bits* depends on input n. The array needs at least $\lceil \log_2 n \rceil$ number of elements. Therefore, the size of the record for storing the local variables of the *main()* function of the program will depend on the value of n. Except for the Stack section of the process, all other sections are independent of n. Therefore, space consumption may be defined as $S(n) = \log_2 n + c = O(\log_2 n)$, which grows logarithmically with n.

Algorithm: Decimal-to-Binary
Input: integer n
Output: binary of n
BEGIN

1.	*Initialize q = n*	1 time
2.	*Initialize i = 0*	1 time
3.	*WHILE q != 0*	$\lceil \log_2 n \rceil$ times
4.	*bits[i] = q%2*	$\lceil \log_2 n \rceil$ times
5.	*q = q/2*	$\lceil \log_2 n \rceil$ times
6.	*i = i + 1*	$\lceil \log_2 n \rceil$ times
7.	*END WHILE*	
8.	*PRINT bits*	$\lceil \log_2 n \rceil$ times – read all the elements in *bits* array

END **Total:** $5\lceil \log_2 n \rceil + 2$

1.1.2 Running time with loop and nested loops

Singh – Lecture notes

From the above examples, it can be seen that the rate of growth of running time of an algorithm depends on the number of iterations in the loops present in the algorithm. Some more cases of loops are discussed in this section. These examples show segments containing the loops (C codes for convenience).

In the example below, let us say, there are few constant time statements before, within and after the *for loop*, and they take constant time *c*. As the loop iterate for n times, its running time can be defined as $T(n) = cn + 2c$, which is a polynomial of degree one. As the value of i in the *for loop* is incremented by one, the statements inside the for loop will be iterated for n times. The iteration can also be mathematically defined as $\sum_{i=1}^{n} c = cn$. Thus, $T(n) = \sum_{i=1}^{n} c + 2c = cn + 2c$.

```
….
for(i=0; i<n; i++){
     [few constant time statements]
}
……
```

In the above estimate, we have not explicitly considered the three statements - $i = 0;$, $i < n;$, and $i++;$ associated with the *for loop* statement. As these statements also contribute to the constant factors, it has been generalized by the constant time factor *c*. Let us consider these statements and also see the estimate. In the for loop, the initialization part $i = 0$ will be executed for 1 time, the comparison part $i < n$ will be executed for $n + 1$ times, and the increment part $i++$ will be executed for n times i.e., a total of $2n + 2$. It means, on an average, two additional statements on top of the statements with the loop in each iteration. As it is yet another constant factor, it does not affect in the asymptotic growth of the algorithm. Therefore, for simplicity, we consider only the number of iterations, if not explicitly specified in the subsequent discussion.

```
….
for(i=1; i<=n; i=i*2){
     [few constant time statements]
}
……
```

In the above example, the value of i is incremented by its double (i.e., $i = i * 2$). Therefore, the *for loop* will iterate for $\log_2 n$ times. The iteration can be mathematically defined as $\sum_{i=1}^{\log_2 n} c = c \log_2 n$. Thus, its running time can be defined as $T(n) = c \log_2 n + 2c$, which is logarithmic.

```
….
for(i=0; i<n; i++){
     for(j=0; j<n; j++)
          [few constant time statements]
}
……
```

In the above example, there is a nested for loop, each loop iterating for n times. The iteration in the nested loop can be defined mathematically as $\sum_{i=1}^{n} \sum_{j=1}^{n} c = cn^2$. Therefore, its running time can be

S Panda – Lecture note

defined as $T(n) = cn^2 + 2c$, which is a polynomial of degree 2. The n^2 in the expression means that statements in the inner for loop will be iterated for n^2 times.

```
….
for(i=0; i<n; i++){
     [few constant time statements]
}
……
for(j=0; j<n; j++){
     [few constant time statements]
}
……
```

In the above example, though there are two for loops, as they are not nested, the effect of the loops will be additive, i.e., $T(n) = cn + cn + 3c = 2cn + 3c$, still a polynomial of degree 1.

Generalization: From the above examples, it can be seen that each loop in an algorithm/program defines another function of n, and the following cases can be realized.

1. If there is one loop defined by $f(n)$, then running time can be defined as $T(n) = f(n) + b$, where b is the constant parameter defined by the statements before and after the loop.
2. If there are a numbers of unnested loops defined by $f(n)$, then running time can be defined as $T(n) = af(n) + b$.
3. If there are a_1 number of loops defined by $f_1(n)$ and a_2 number of loops defined by $f_2(n)$, then running time can be defined as $T(n) = a_1 f_1(n) + a_2 f_2(n) + b$.
4. A nested loop of $f(n)$ within another loop $g(n)$ can be defined as $T(n) = g(f(n)) + b$.
5. Similarly, k number of deep nested loops can be defined as $T(n) = f_1\left(f_2\left(f_3(\dots f_k(n))\right)\right) + b$.

1.1.3 Best, Average and Worst Cases

For the examples such as factorial or integer to binary conversion, there is only one input scenario i.e., the given value n. However, for some classes of problem, there could be multiple input scenarios, and the running time of the algorithm will be different for different input scenarios. An example of such problems is linear search. *Given an array of elements, the task is to check the occurrence of an element*. For searching an element in the given array, the elements in the array need to be checked linearly one-by-one until the target element is found or all the elements are checked. One such algorithm is given below.

Algorithm: LinearSearch
Input: integer e, integer array A
Output: Print the index of the element if FOUND
Assumption: A has n elements
BEGIN

1.	*Initialize i = 0, loc = -1*	*1 time for two statements*
2.	*WHILE i < n*	*upto n+1 times*
3.	*IF A[i] == e THEN*	*upto n times*
4.	*i = i + 1*	*1 time*
5.	*Break*	
6.	*END IF*	
7.	*END WHILE*	
8.	*IF i != -1 THEN*	*1 time*
9.	*PRINT FOUND at i-1*	*1 time*
10.	*ELSE*	
11.	*Print NOT FOUND*	*1 time*

Prabhjot Singh - Lecture note

For this problem, we have the following three cases.

Best Case: The element to be searched is the first element of the array. In this case, the WHILE loop iterates only one time, and $T(n) = \sum_{i=1}^{1} c + d = c + d$, where c is the constant number of steps within the loop, and d is the constant number of steps before and after the loop. Best case takes constant running time.

Worse Case: The worst case occurs when the element is not found (or the element is the last element of the array). If the element is not found, the running time is $T(n) = \sum_{i=1}^{n} c + d$, which is a polynomial of degree 1. That means, running time linearly increases with the size of the array.

Average Case: In average case analysis, it is assumed that the element to be searched may occur at any position in array with equal probability. If the element is present at the ith position, the number of iterations of the while loop is i, i.e., $\sum_{j=1}^{i} c$. If $Pr(i)$ is the probability of occurring the searched element at the position i, then its running time can be defined as $\Pr(i) \sum_{j=1}^{i} c = \frac{1}{n} \sum_{j=1}^{i} c$, where $Pr(i) = \frac{1}{n}$. So, the expected average case running time can be defined as follow.

$$T(n) = \frac{1}{n}\sum_{j=1}^{1} c + \frac{1}{n}\sum_{j=1}^{2} c + \frac{1}{n}\sum_{j=1}^{3} c + \cdots . + \frac{1}{n}\sum_{j=1}^{n} c + d$$

$$T(n) = c\frac{1}{n} + 2c\frac{1}{n} + 3c\frac{1}{n} + \cdots . + nc\frac{1}{n} + d = \frac{n(n+1)c}{2n} + d = \frac{(n+1)c}{2} + d$$

The "+ d" in the above expression is for the constant steps executed before and after the while loop. The above average running time is also a polynomial of degree one.

Let us take another example of sorting elements in an array to understand another scenario of best, average and worst cases. The given algorithm is known as *bubble sort*, which is discussed in detailed in unit 4 of this book. Given an array, the algorithm will sort the elements in the array in ascending order.

```
Algorithm: bubbleSort
Input: Array a
Output: Sorted array A
BEGIN
1.   FOR (i = 0; i < n - 1; i++)
2.     FOR (j = 0; j < n - i - 1; j++)
3.       IF (a[j] > a[j + 1])
4.         swap(a[j], a[j + 1])   //a function to exchange the values at a[j] and a[j+1]
5.       END IF
6.     END FOR
7.   END FOR
END
```

In the above algorithm, the outer loop iterates for $n - 1$ times and the inner loop iterates for $n - i$ times. If the time for the constant steps inside the inner loop is c, then running time of the nested loop can be defined as $T(n) = \sum_{i=1}^{n-1} \sum_{j=i}^{n-i} c + d = c \sum_{i=1}^{n-1} (n - i) + d = c((n-1) + (n-2) + (n -$

$3)+..+3+2+1)+d = c\frac{n(n-1)}{2}+d$, where d is for any other constant steps if any. It is a polynomial of degree two.

In the above algorithm, for the cases where $(a[j] > a[j+1])$ is false, the swap operation is ignored. If the given array is already in sorted order, the condition $(a[j] > a[j+1])$ will always be false, and no swap operation will be performed. However, in the given algorithm above, as we do not check for the case of already sorted input array, irrespective of the ordering of the elements in the input array, both the outer and inner loop will iterate for their complete cycles and perform $(a[j] > a[j+1])$ comparison for $\frac{n(n-1)}{2}$ times. As the number of comparison operations dominates swap operations, we consider comparison operation as the basic operation inside the loop.

Can we do better? In the algorithm given above, let us assume that for a given i in the outer loop, the condition $(a[j] > a[j+1])$ inside the inner loop always false, i.e., no swap operation happens for a given i in the outer loop. It indicates that all the elements in the array are already in order at the time of processing of the inner loop for the given i. At this point, if we terminate the algorithm, we can avoid un-necessary iteration for the remaining i. The following algorithm modifies the above algorithm to check, if the elements in the array are already in sorted order, before completing the outer loop. A flag `swapped` is set to 0 before the inner loop. If any consecutive pair of elements are not in ordered, the inner loop will set the flag `swapped` to 1. After completing the inner loop for a given i, if the value of `swapped` is 0, it means that the array is already in sorted order. If the part of the array to be processed by the inner loop is in already sorted order, the algorithm terminates. With this modification, we have the following three cases.

Algorithm: bubbleSortModified
Input: Array a
Output: Sorted array A

```
BEGIN
1.   FOR (i = 0; i < n-1; i++)
2.      swapped = 0;
3.      FOR (j = 0; j < n-i-1; j++)
4.         IF (a[j] > a[j+1])
5.            swap(a[j], a[j+1]);
6.                     swapped = 1;
7.         END IF
8.      END FOR
9.     IF (swapped == 0)
10.       break;
11.   END IF
12. END FOR
END
```

Best Case: The elements in the given array are already in ascending order. In this scenario, the outer loop will iterate only one time, and inner loop will iterate for n-1 times without performing any swap operation. So, running time of this algorithm can be expressed as $T(n) = (n-1)c + d$, which is a polynomial of degree one. It means, the algorithm performs n-1 number of comparison operations.

Worst Case: The elements in the given array are in reverse order i.e., descending order. In this scenario, the outer loop will iterate for $n-1$ times, and for each value of i in outer loop, the inner loop iterates for $n-i$ times. For every inner loop iteration, $n-i$ number of swap operations are performed. So, the running time of the algorithm for this case can be expressed as $T(n) = \sum_{i=1}^{n-1}\sum_{j=i}^{n-i} c + d = c\frac{n(n-1)}{2} + d$, which is a polynomial of degree two. It means, the algorithm performs $\frac{n(n-1)}{2}$ number of comparison operations (also swap operations).

Average Case: The elements in the given array are in any random order. Without proving mathematically, we can roughly say, in average case, there are about half the number of swap operation as that of the worst case, that is, $\frac{n(n-1)}{4}$ number of swap operations on average. In other words, in worst case, we have seen that $n-1$ number of swap operations are performed for the last index in the sorted array, $n-2$ number of swap operations for the last but one index, and so on. So, on an average, from the first index till the last applicable index, there are $\frac{1}{2}+\frac{2}{2}+\frac{3}{2}+\cdots+\frac{n-3}{2}+\frac{n-2}{2}+\frac{n-1}{2}=\frac{n(n-1)}{4}$ number of swap operations. Therefore, average case running time for bubble sort algorithm is also a polynomial of degree two.

1.2 ASYMPTOTIC NOTATIONS

In the above discussion, the running time and space consumption have been defined as a function of n i.e., $T(n)$ for running time and $S(n)$ for space consumption. As mentioned in section 1.2.5, the time complexity and space complexity are represented using *Asymptotic Notations* ($\theta, O, o, \Omega, \omega$). As we are interested in the rate of growth, the idea of using asymptotic notations is to represent complexity in terms of a prior known class of rate of growth such as $\log_2 n, n, n\log_2 n, n^2, n^3, 2^n$ etc. Readers may refer *Introduction to Algorithms, by Thomas Cormen, Charles Leiserson, Rolard Rivest, and Clifford Stein* for extended study on the analysis of algorithms.

1.2.1 Big Oh Notation (O)

Definition: Let $f(n)$ be the function defining efficiency of an algorithm, say $T(n)$ or $S(n)$ above. Now, $f(n)$ can be written as $O(g(n))$ i.e., $f(n)=O(g(n))$ if and only if there exist two positive constants C and n_0 such that $0 \le f(n) \le Cg(n), \forall n \ge n_0$.

Let us illustrate the above definition with the following example. Let $f(n)=2n+4$ and $g(n)=n$. To prove $f(n)=O(g(n))$, we just need to find the existence of any C and n_0 satisfying the condition $f(n) \le Cg(n), \forall n \ge n_0$. Let us assume $C=3$, and estimate $f(n)$ and $Cg(n)$ for different values of n, as shown in the following table.

Different values of n	$f(n)=2n+4$	$Cg(n)=3n$	$f(n) \le Cg(n)$
1	6	3	No
2	8	6	No
3	10	9	No
4	12	12	For all values of $n \ge 4$, the condition $f(n) \le Cg(n)$ holds. So, n_0 is 4.
5	14	15	
6	16	18	

For $C=3$ and $n_0=4$, as the condition $f(n) \le Cn$ holds, $f(n)$ can be written as $O(n)$ i.e., $f(n)=O(n)$. *What about* $g(n)=n^2$*?* For the same value of C, we can estimate the following.

Different values of n	$f(n)=2n+4$	$Cg(n)=3n^2$	$f(n) \le Cg(n)$
1	6	3	No
2	8	12	For all values of $n \ge 2$, the condition $f(n) \le Cg(n)$ holds. So, n_0 is 2.
3	10	27	
4	12	48	
5	14	75	

S Ranbir Singh - Lecture note

6	16	108	

For $C = 3$ and $n_0 = 2$, as the condition $f(n) \leq Cn^2$ holds, $f(n)$ can also be written as $O(n^2)$ i.e., $f(n) = O(n^2)$. Without loss of generality, we can see that $f(n)$ holds for all higher degrees of n, such as $O(n \log n), O(n^2), O(n^2 \log n), O(n^3)$ etc. Among these possible $g(n)$, the $O(n)$ is the tight most one. Therefore, we generally say $f(n) = 2n + 4 = O(n)$. *What about* $g(n) = \log n$*?* Without difficulty, it can be seen that for any value of C, the condition $f(n) \leq Cg(n)$ may not hold for large value of n. Therefore, $f(n) \neq O(\log n)$. Similarly, $f(n) \neq O(1)$. The $O(1)$ means *asymptotic constant*. For example, $T(n) = 3 = O(1)$.

In the approach of finding asymptotic complexity of an algorithm defined by a function of *n*, we first *guess a bound and then prove that the guess is correct.* This way of finding the boundary function is known as the *substitution method.*

Remarks: As the definition indicates, Big Oh says that the growth of $f(n)$ is always smaller or equal to that of $g(n)$. Thus, $g(n)$ defines an *upper bound* of $f(n)$. Therefore, Big Oh may or may not represent an asymptotically tight upper bound. For example, the bound $2n + 3 = O(n)$ is tight, but $2n + 3 = O(n^2)$ is not tight.

1.2.2 Big Omega Notation (Ω)

Definition: Let $f(n)$ be the function defining efficiency of an algorithm. Then, $f(n)$ can be written as $\Omega(g(n))$ i.e., $f(n) = \Omega(g(n))$ if and only if there exist two positive constants C and n_0 such that $0 \leq f(n) \geq Cg(n), \forall n \geq n_0$.

Let $f(n) = 2n + 4$ and $g(n) = n$. To prove $f(n) = \Omega(g(n))$, we just need to find existence of any C and n_0 satisfying the condition $f(n) \geq Cg(n), \forall n \geq n_0$. Let us assume $C = 2$, and estimate $f(n)$ and $Cg(n)$ for different values of n as shown in the following table.

Different values of n	$f(n) = 2n + 4$	$Cg(n) = 2n$	$f(n) \leq Cg(n)$
0	4	0	No
1	6	2	For all values of $n \geq 1$, the condition $f(n) \geq Cg(n)$ holds. So, n_0 is 1.
2	8	4	
3	10	6	
4	12	8	
5	14	10	
6	16	12	

For all values of $C = 2$ and $n_0 = 1$, as the condition $f(n) \geq Cn$ holds, $f(n) = \Omega(n)$. Similarly, $f(n)$ will hold for lower degrees $\Omega(1)$ and $\Omega(\log n)$, but **will not hold for higher degrees** such as $\Omega(n \log n), \Omega(n^2), \Omega(n^2 \log n), \Omega(n^3)$ etc.

Remarks: As the definition indicates, Big Omega says that the growth of $f(n)$ is always greater or equal to that of $g(n)$. Thus, $g(n)$ defines a *lower bound* of $f(n)$. Therefore, Big Oh may or may not represent an asymptotically tight upper bound. For example, the bound $2n + 3 = \Omega(n)$ is tight, but $2n + 3 = O(\log n)$ is not tight.

1.2.3 Theta Notation (θ)

Definition: Let $f(n)$ be the function defining efficiency of an algorithm. Then, $f(n)$ can be written as $\boldsymbol{\theta}(g(n))$ i.e., $f(n) = \boldsymbol{\theta}(g(n))$ if and only if there exist three positive constants C_1, C_2 and n_0 such that $0 \leq C_1 g(n) \leq f(n) \leq C_2 g(n), \forall n \geq n_0$.

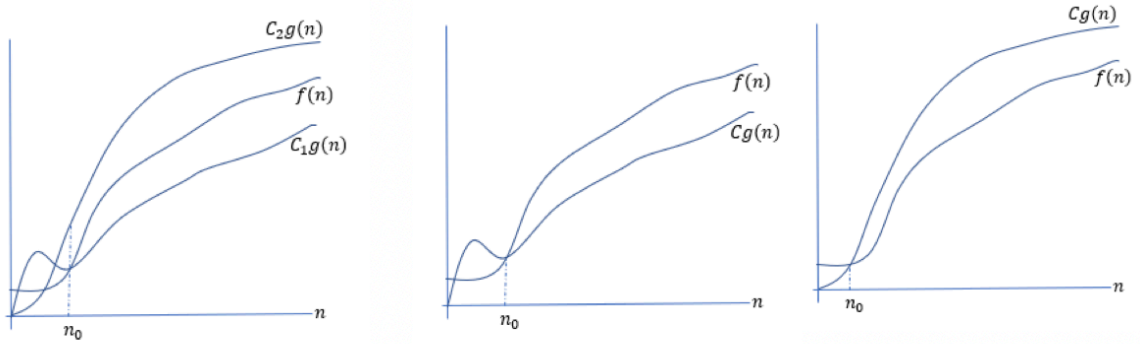
S Ranbir Singh - Lecture note

From the above examples, it can be seen that $f(n) = 2n + 4$ can be written as $\theta(n)$. Let $C_1 = 2$ and $C_2 = 3$, and estimate as follows

Values of n	$f(n) = 2n + 4$	$C_1 g(n) = 2n$	$C_2 g(n) = 3n$	$C_1 g(n) \leq f(n) \leq C_2 g(n)$
0	4	0	0	No
1	6	2	3	No
2	8	4	6	No
3	10	6	9	No
4	12	8	12	For all values of $n \geq 4$, the condition holds. So, $f(n) = \theta(n)$.
5	14	10	15	
6	16	12	18	

Therefore, $f(n) = 2n + 4 = \theta(n)$. From the above examples, it can be seen that $f(n) = \theta(g(n))$ if and only if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

Remarks: As the definition indicates, Theta says that the growth of $f(n)$ should not be smaller or greater than that of $g(n)$. Thus, $g(n)$ defines a *tight bound* of $f(n)$. Figure 1.7 presents an example illustrating relationship between $f(n)$ and $g(n)$ for different asymptotic notations.



(a) $f(n) = \theta(g(n))$ (b) $f(n) = \Omega(g(n))$ (c) $f(n) = O(g(n))$

Figure 2: An illustration of the relationship between f(n) and g(n) for different asymptotic notations.

Further, the following table shows the rate of growth for some of the common functions with approximate values. It clearly shows that the growth rates of these functions hold the following relationships.

$$1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$$

input	1	$\log_2 n$	n	$n\log_2 n$	n^2	n^3	2^n	$n!$
10	1	3.3	10	3.3×10	10^2	10^3	10^3	3.6×10^6
10^2	1	6.6	10^2	6.6×10^2	10^4	10^6	1.3×10^{30}	3.6×10^{157}
10^3	1	10	10^3	10^4	10^6	10^9		
10^4	1	13	10^4	13×10^4	10^8	10^{12}		
10^5	1	17	10^5	17×10^5	10^{10}	10^{15}		
10^6	1	20	10^6	20×10^6	10^{12}	10^{18}		

Table 1.1: Approximate values of some of the functions for different values of n

S Ranbir Lecture note

1.2.4 Small Oh and Small Omega Notations (o, ω)

Definition Small Oh (o): A $f(n)$ can be written as $o(g(n))$ i.e., $f(n) = o(g(n))$ if and only if for any positive constants C, there exists a positive constant n_0 such that $0 \leq f(n) < Cg(n), \forall n \geq n_0$. That is, $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$.

In the case of Big Oh, the condition $f(n) < Cg(n)$ should hold for a particular C, whereas the condition should hold for all C in the case of small oh. Therefore, $f(n) = 2n + 4 = O(n)$, but $f(n) = 2n + 4 \neq o(n)$ as the condition does not hold for C<=2. Thus, $f(n) = 2n + 4 = o(n \log n)$ or $f(n) = 2n + 4 = o(n^2)$. In this, $g(n)$ defines a loose bound $f(n)$.

Definition Small Omega (ω): A $f(n)$ can be written as $\omega(g(n))$ i.e., $f(n) = \omega(g(n))$ if and only if for any positive constants C, there exists a positive constant n_0 such that $0 \leq f(n) > Cg(n), \forall n \geq n_0$. That is, $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$.

Analogy to definition of Big Oh and small oh, $f(n) = 2n + 4 = \Omega(n)$, but $f(n) = 2n + 4 \neq \omega(n)$ as the condition does not hold for C>=5. Thus $f(n) = 2n + 4 = \omega(\log n)$, or $f(n) = 2n + 4 = \omega(1)$. In this, $g(n)$ defines a loose bound $f(n)$.

1.2.5 Few Asymptotic Properties

If $f(n)$, g(n) and $h(n)$ are asymptotically positive functions, then the following properties hold.

Reflexive:

- $f(n) = O(f(n))$
- $f(n) = \theta(f(n))$
- $f(n) = \Omega(f(n))$

Symmetric:

- $f(n) = \theta(g(n))$ if and only if $g(n) = \theta(f(n))$

Transitive:

- If $f(n) = \theta(g(n))$ and $g(n) = \theta(h(n))$, then $f(n) = \theta(h(n))$
- If $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$
- If $f(n) = \Omega(g(n))$ and $g(n) = \Omega(h(n))$, then $f(n) = \Omega(h(n))$
- If $f(n) = o(g(n))$ and $g(n) = o(h(n))$, then $f(n) = o(h(n))$
- If $f(n) = \omega(g(n))$ and $g(n) = \omega(h(n))$, then $f(n) = \omega(h(n))$

Transpose Symmetric:

- $f(n) = O(g(n))$ if and only if $g(n) = \Omega(f(n))$
- $f(n) = o(g(n))$ if and only if $g(n) = \omega(f(n))$

1.2.6 Solving Recurrence Equation

Running time of a recursive algorithm can be represented by a *recurrence expression*. In section 1.4, we have taken an example of estimating the factorial of n using a recursive algorithm, whose running time can be represented using a recurrence expression $T(n) = T(n-1) + c, T(1) = 1$. We have expanded the expression to obtain the expression $T(n) = cn$. Then, by substitution, we can prove that $T(n) = \theta(n)$. Some more examples of recurrence expression and their solutions using substitution methods are discussed.

S Ranbir Singh - Lecture note

Example 1.13: $T(n) = \begin{cases} 2T(n-1)+1, & if\ n > 1 \\ 1 & , \ if\ n = 1 \end{cases}$

Description: The algorithm has two recursive calls with n-1 data size

$$T(n) = 2T(n-1) + 1 = 2[2T(n-2) + 1] + 1 = 2^2T(n-2) + 2^1 + 2^0$$

$$= 2^2[2T(n-3) + 1] + 2^1 + 2^0 = 2^3T(n-3) + 2^2 + 2^1 + 2^0$$

$$= 2^3[2T(n-4) + 1] + 2^2 + 2^1 + 2^0 = 2^4T(n-3) + 2^3 + 2^2 + 2^1 + 2^0$$

…….

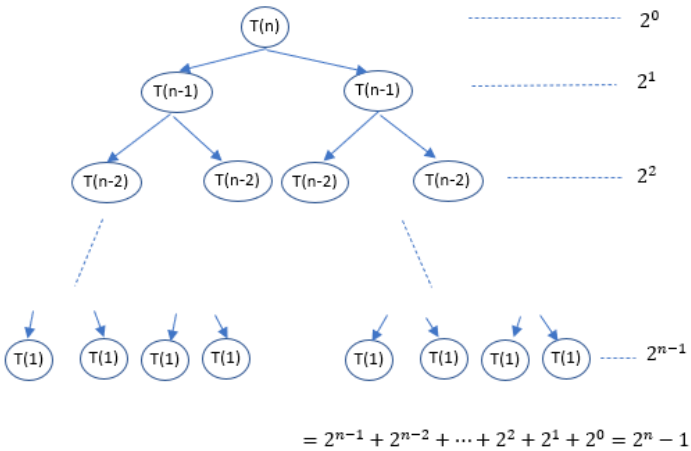
……..

$$= 2^{n-1}T(1) + 2^{n-2} + \cdots + 2^3 + 2^2 + 2^1 + 2^0$$

$$= 2^{n-1} + 2^{n-2} + \cdots + 2^3 + 2^2 + 2^1 + 2^0$$

$$= 2^n - 1 = O(2^n)$$

Execution of above recurrence expression can also be visualized in the form of a binary tree (a node in the tree can have at most two children). $T(n) = 2T(n-1) + 1$ means, the task of solving the problem is divided into two sub-tasks with size reduced by one, and the cost of dividing the task and other constant computations is 1. Running time can also be estimated from the figure as well as given below.



$$= 2^{n-1} + 2^{n-2} + \cdots + 2^2 + 2^1 + 2^0 = 2^n - 1$$

Each node in the above tree represents a function call, and two children represent two instances of recursive calls from each function. At level $i \geq 0$ of the tree, there are 2^i number of nodes, and each node consumes a constant computation cost of 1. Therefore, level i of the tree consumes a computation cost of 2^i. There are n number of levels. Therefore, the total cost is $2^{n-1} + 2^{n-2} + \cdots + 2^2 + 2^1 + 2^0 = O(2^n)$. The estimation using the above tree approach is known as *Recursion Tree Method*.

S Ran… Lecture note

Example 1.14: $T(n) = \begin{cases} 2T(n-1)+3, & if\ n > 1 \\ 1 & ,\ if\ n = 1 \end{cases}$

Description: The algorithm has two recursive calls with n-1 data size

$$T(n) = 2T(n-1)+3 = 2[2T(n-2)+3]+3 = 2^2T(n-2)+3.2^1+3.2^0$$

$$= 2^2[2T(n-3)+3]+3.2^1+3.2^0 = 2^3T(n-3)+3.2^2+3.2^1+3.2^0$$

$$= 2^3[2T(n-4)+3]+3.2^2+3.2^1+3.2^0 = 2^4T(n-3)+3.2^3+3.2^2+3.2^1+3.2^0$$

……

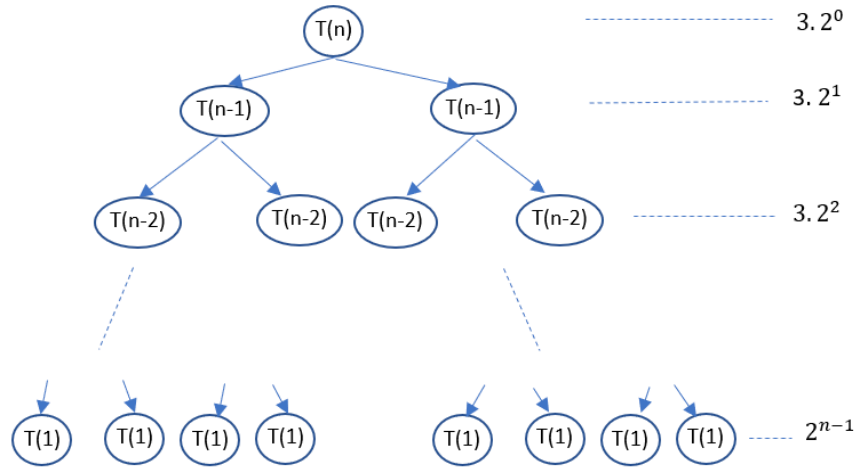
……..

$$= 2^{n-1}T(1)+3.2^{n-2}+\cdots+3.2^2+3.2^2+3.2^1+3.2^0$$

$$\approx 3.2^{n-1}+3.2^{n-2}+\cdots+3.2^2+3.2^2+3.2^1+3.2^0$$

$$\approx 3(2^n-1) = O(2^n)$$

Using recursion tree method, the running time of $T(n) = 2T(n-1)+3$ can be estimated as follows. The computation cost at i^{th}level is 3×2^i, and there are n number of levels.



$$= 2^{n-1}+3.2^{n-2}+\cdots+3.2^2+3.2^1+3.2^0 \approx 3.2^n-1$$

Example 1.15: $T(n) = \begin{cases} 2T(n/2)+1, & if\ n > 1 \\ 1 & ,\ if\ n = 1 \end{cases}$

Description: The algorithm has two recursive calls with n/2 data size

$$T(n) = 2T(n/2)+1 = 2[2T(n/2^2)+1]+1 = 2^2T(n/2^2)+2^1+2^0$$

$$= 2^2[2T(n/2^3)+1]+2^1+2^0 = 2^3T(n/2^2)+2^2+2^1+2^0$$

$$= 2^3[2T(n/2^4)+1]+2^2+2^1+2^0 = 2^4T(n/2^4)+2^3+2^2+2^1+2^0$$

Lecture note

……

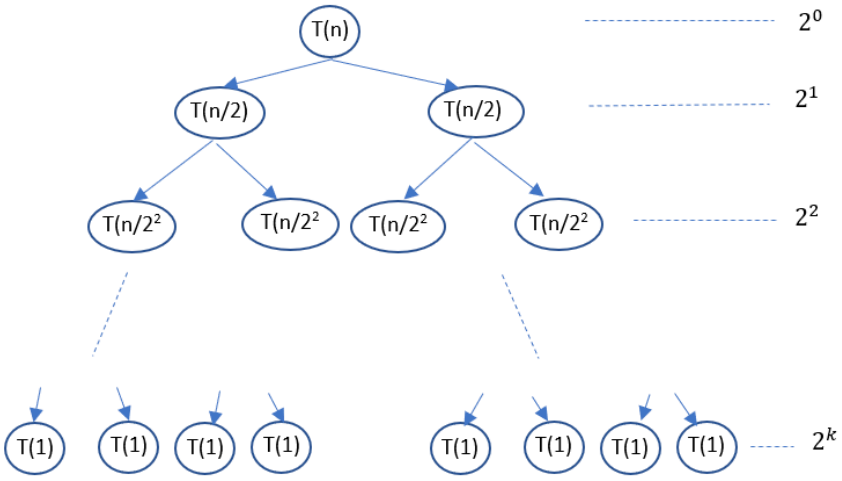
……

$= 2^k T(n/2^k) + 2^{k-1} + \cdots + 2^2 + 2^2 + 2^1 + 2^0$

$= 2^k T(1) + 2^{k-1} + \cdots + 2^2 + 2^2 + 2^1 + 2^0$, *Assume that* $n = 2^k$

$= 2^k T(1) + 2^{k-1} + \cdots + 2^2 + 2^2 + 2^1 + 2^0 = 2^{k+1} - 1 = 2^k + 1 = n + 1 = O(n)$

Using recursion tree method, the running time of can be estimated as follows. The computation cost at i^{th}level is 2^i, and there are $\log_2 n$ number of levels.



$= 2^k + 2^{n-1} + \cdots + 2^2 + 2^1 + 2^0 = 2^k + 1 = n + 1$

Example 1.16: $T(n) = \begin{cases} 2T(n-1) + n, & if\ n > 1 \\ 1, & if\ n = 1 \end{cases}$

Description: The algorithm has two recursive calls with n-1 data size

$T(n) = 2T(n-1) + n = 2[2T(n-2) + (n-1)] + n = 2^2 T(n-2) + (n-1)2^1 + n2^0$

$= 2^2[2T(n-3) + (n-2)] + (n-1)2^1 + n.2^0$
$= 2^3 T(n-3) + (n-2)2^2 + (n-1)2^1 + n2^0$

$= 2^4 T(n-4) + (n-3)2^3 + (n-2)2^2 + (n-1)2^1 + n2^0$

……

……

$= 2^{n-1} T(1) + (n-n-2)2^{n-2} + \cdots + (n-3)2^3 + (n-2)2^2 + (n-1)2^1 + n2^0$

$\approx (n-(n-1))2^{n-1} + (n-(n-2))2^{n-2} + \cdots + (n-3)2^3 + (n-2)2^2 + (n-1)2^1 + (n-0)2^0$

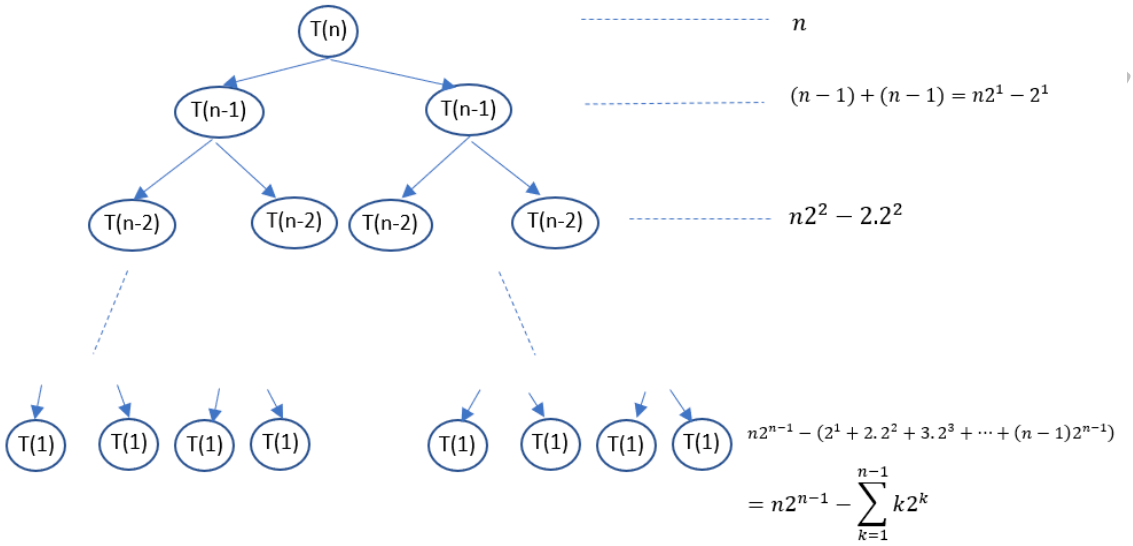
$= n(2^{n-1} + 2^{n-2} + \cdots + 2^3 + 2^2 + 2^1 + 2^0) - [(n-1)2^{n-1} + (n-2)2^{n-2} + \cdots + 3.2^3 + 2.2^2 + 1.2^1]$

S Ranbir Singh Note

$= n(2^n - 2) - 2(1 - n2^n + (n-1)2^n)$ as $\sum_{k=1}^{n} kz^k = z\frac{1-(n+1)z^n+nz^{n+1}}{(1-z)^2}$

$= O(n2^n)$

Using recursion tree method, the running time of $T(n) = 2T(n-1) + n$ can be estimated as follows. The computation cost at i^{th} level is $n2^{i-1} - (i-1).2^{i-1}, i \geq 1$, and there are n number of levels.



Example 1.17: $T(n) = \begin{cases} 2T(n/2) + n, & if\ n > 1 \\ 1 & , if\ n = 1 \end{cases}$

Description: The algorithm has two recursive calls with n-1 data size

$T(n) = 2T(n/2) + n = 2[2T(n/2^2) + n/2] + n = 2^2T(n/2^2) + n + n$

$= 2^2[2T(n/2^3) + n/4] + n + n = 2^3T(n/2^2) + n + n + n$

$= 2^3[2T(n/2^4) + n/8] + n + n + n = 2^4T(n/2^4) + n + n + n + n$

……

……

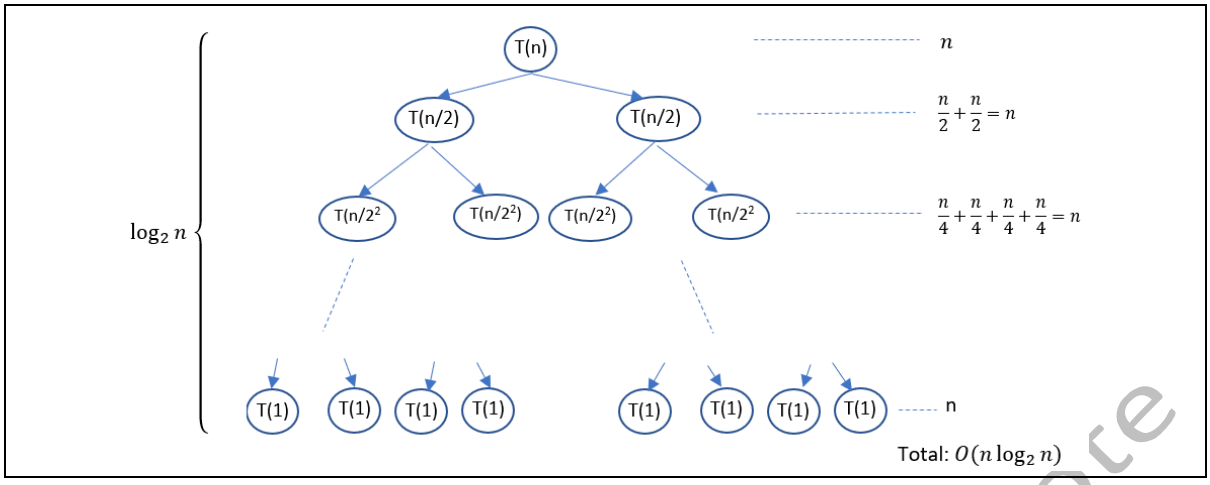
$= 2^kT(n/2^k) + n + \cdots + n + n + n + n$

$= 2^kT(1) + n + \cdots + n + n + n + n$, *Assume that* $n = 2^k$

$= n\log_2 n + n = O(n\log_2 n)$

Using recursion tree method, the running time of $T(n) = 2T(n/2) + n$ can be estimated as follows. The computation cost at every level is n, and there are $\log_2 n$ number of levels.

S Panbir Singh -



From the above examples, it can be seen that for solving a recursive algorithm, one needs to expand its recurrence expression to a standard series to get the corresponding asymptotic notation. However, it may not be always possible or trivial to get such an expression. In such as case, one needs to approximate with a known bound. Let us consider another algorithm for generating *Fibonacci series i.e., 0, 1, 1, 2, 3, 5, ...*, where a number in the series is the sum of the previous two, i.e., $F_n = F_{n-1} + F_{n-2}, F_0 = 0$ and $F_1 = 1$. We can write both the iterative and recursive algorithms for generation first n Fibonacci series as follows.

Name: FibonacciSeries
Input: Integer number n
Output: First n Fibonacci series
Description: $F_n = F_{n-1} + F_{n-2}, F_0 = 0, F_1 = 1$

```
BEGIN
1.  Initialize F1 = 0, F2=1 and i=2
2.  PRINT F1, F2
3.  WHILE (i < n)
4.        F3 = F1 + F2
5.        PRINT F3
6.        F1=F2
7.        F2=F3
8.        i = i+1
9.  END WHILE
END
```

Name: Fib (n)
Input: Integer number n
Output: First n Fibonacci series
Description: F(n)=F(n-1)+F(n-2)

```
BEGIN
1.  IF n==1 OR n==0 THEN
2.      PRINT n
3.      RETURN n
4.  ELSE
5.      PRINT Fib(n-1)+ Fib (n-2)
6.  END IF
END
```

Without difficulty, it can be seen that the iterative algorithm has a linear bound i.e., O(n), as the while loop iterates for n times and there are constant running time steps within the while loop. However, the recursive algorithm has a recursive expression of the form $T(n) = T(n-1) + T(n-2) + c$. Because, it has two recursive calls of the size n-1 and n-2, and a constant computation time for splitting the task and other constant steps. If we expand the recurrence expression, we may not be able to get a standard series easily. For simplicity, let us assume $c = 1$.

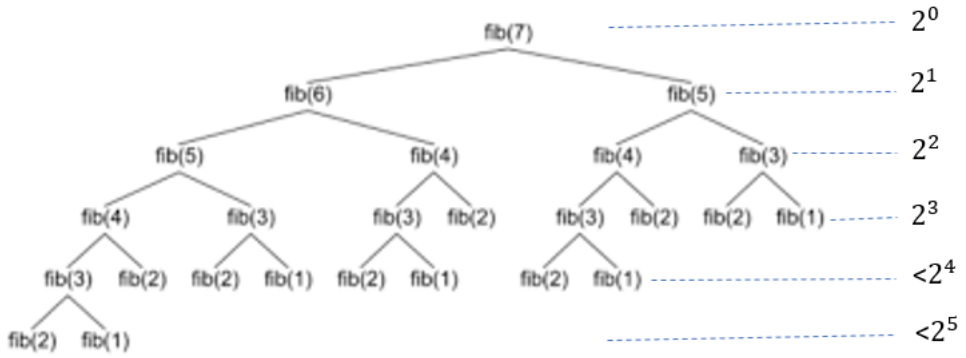
$$T(n) = T(n-1) + T(n-2) + 1$$
$$= T(n-2) + T(n-3) + 1 + T(n-3) + T(n-4) + 1 + 1$$
$$= T(n-2) + 2T(n-3) + T(n-4) + 3$$
$$= T(n-3) + T(n-4) + 1 + 2T(n-4) + 2T(n-5) + 2 + T(n-5) + T(n-6) + 1 + 3$$
$$= T(n-3) + 3T(n-4) + 3T(n-5) + T(n-6) + 1 + 2 + 1 + 3$$

....

.....

The above expression needs to be expanded till every recursive call reduces to T(1). It is not easy to converse to a standard expression/series. In such a scenario, one can approximate with a known upper bound to get its asymptotic notation. It may be easier to use a pictorial tree-based expansion to approximate its asymptotic relation.

From the Figure 1.8, which computes fib(7), it can be seen that the nodes in a level is complete only upto 4^{th}level. That means, given n, only upto $\lceil n/2 \rceil$ levels will be complete, the rest of the levels will be incomplete. However, as it is a binary tree, we know that the number of nodes cannot be more than 2^i. So, we can assume the bound as 2^i, and approximate the series as $T(n) < 2^{n-2} + 2^{n-1} + \cdots + 2^2 + 2^1 + 2^0 \approx 2^{n-1} - 1$, and obtain the asymptotic complexity as $O(2^n)$. The assuming 2^n bound above may be a loose bound, as we know that the last levels will have nodes lesser that 2^n. The reader may attempt to get a better bound.



$$T(n) < 2^{n-2} + 2^{n-1} + \cdots + 2^2 + 2^1 + 2^0 \approx 2^{n-1} - 1$$

Figure 3: Tree representation of time complexity computation for fib(7)

What about the space complexity? As seen in Figure 1.7, the height of the tree is at the most $n - 1$. It means that the stack sector of the process running this algorithm will grow at the most by $n - 1$ records. At any instance in time, the stack will hold at the most $n - 1$ number of records of the recursive function. Therefore, space complexity is $O(n)$.

Similarly, let us take another example $T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + n$. It divides the task into two, but with unequal sizes. Figure 1.8 illustrates the recursive expansion.

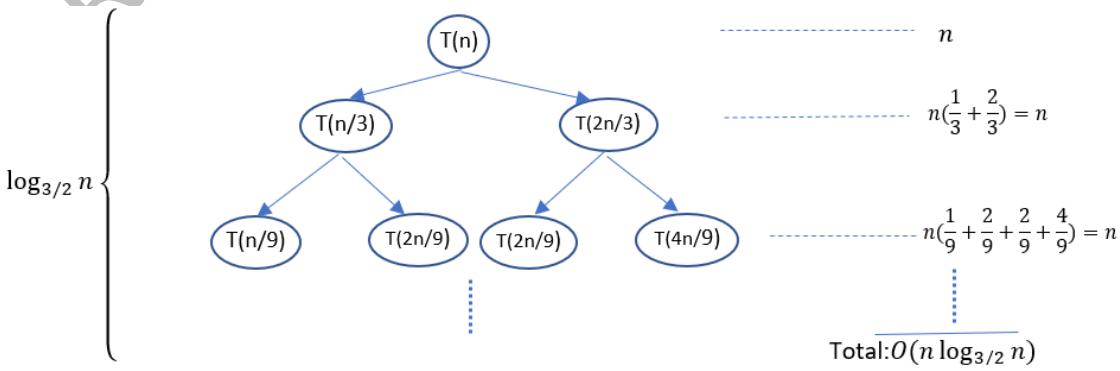


Figure 4: Recussive expansion for T(n)=T(n/3)+T(2n/3)+n

Ranbir Singh

Every level has n computational cost and there are $\log_{3/2} n$ levels. Therefore, asymptotic complexity is $O(n \log_{3/2} n)$. It will have the space complexity of $O(\log_{3/2} n)$, i.e., the height of the tree.

1.2.7 Master Theorem

As discussed in the classic book, *Introduction to Algorithms, by Thomas Cormen, Charles Leiserson, Rolard Rivest, and Clifford Stein, the* asymptotic complexity of some of the standard recurrence expressions can be directly obtained using Master Theorm, which is briefly defined below.

Let $T(n)$ be a recurrence expression of the form $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ defined over a non-negartive integer n, where $a \geq 1$ and $b > 1$ are constent, and $f(n)$ is a function of n. The $\frac{n}{b}$ in the recurrence expression represents either $\left\lfloor \frac{n}{b} \right\rfloor$ or $\left\lceil \frac{n}{b} \right\rceil$. Then, the $T(n)$ has the following asymptotic complexity bounds.

1. If $f(n) = O(n^{\log_b a - \varepsilon})$ for some constant $\varepsilon > 0$, then $T(n) = \theta(n^{\log_b a})$.
2. If $f(n) = \theta(n^{\log_b a})$ for some constant $\varepsilon > 0$, then $T(n) = \theta(n^{\log_b a} \lg n)$.
3. If $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some constant $\varepsilon > 0$, and if $af\left(\frac{n}{b}\right) \leq cf(n)$ for all sufficiently large n and some constant $c < 1$, then $T(n) = \theta(f(n))$.

In all the above three cases, the function $f(n)$ is compared with the function $n^{\log_b a}$. If one if asymptotically larger than the another, the asymptotic complexity of $T(n)$ is defined as θ of the larger function. If they are same, then multiply $T(n)$ by $\lg n$. In case 1, $n^{\log_b a}$ is asymptotically larger than $f(n)$, so, $T(n) = \theta(n^{\log_b a})$. In case 3, $f(n)$ is asymptotically larger than $n^{\log_b a}$, so $T(n) = \theta\big(f(n)\big)$. In case 2, they are asymptotically same, so $T(n) = \theta(\lg n\, f(n))$.

Let us take the following examples to understand the above conditions.

1. For $T(n) = T\left(\frac{n}{2}\right) + 1, a = 1, b = 2$ and $f(n) = 1$. Now, $f(n) = n^{\log_b a} = n^{\log_2 1} = 1$. Therefore, $T(n) = \theta\big(\lg n\, n^{\log_2 1}\big) = \theta(\lg n)$. If you recursively expand the expression $T(n) = T\left(\frac{n}{2}\right) + 1$ with $T(1) = 1$, you will also see that $T(n) = \log_2 n + 1 = \theta(\log_2 n)$.
2. For $T(n) = 2T\left(\frac{n}{2}\right) + 1,\ a = 2, b = 2$ and $f(n) \leq n^{\log_b a - \varepsilon} = n^{\log_2 2} = n$ for $\varepsilon = 0$. Therefore $T(n) = \theta\big(n^{\log_2 2}\big) = \theta(n)$. You have also seen in Example 1.15 that $T(n) = 2T\left(\frac{n}{2}\right) + 1 = \theta(n)$.
3. For $T(n) = 2T\left(\frac{n}{2}\right) + n,\ a = 2, b = 2$ and $f(n) = n^{\log_b a} = n^{\log_2 2} = n$. Therefore $T(n) = \theta\big(\lg n\, n^{\log_2 2}\big) = \theta(n \lg n)$. You have also seen in Example 1.17 that $T(n) = 2T\left(\frac{n}{2}\right) + n = \theta(n \log_2 n)$.
4. For $T(n) = 2T\left(\frac{n}{2}\right) + n^2, a = 2, b = 2$ and $f(n) \geq n^{\log_b a + \varepsilon} = n^{\log_2 2} = n$ for $\varepsilon = 0$. Therefore, $f(n) = \Omega\big(n^{\log_2 2}\big) = \Omega(n)$, and $T(n) = \theta(f(n)) = \theta(n^2)$.
5. For $T(n) = 5T\left(\frac{n}{2}\right) + n,\ a = 5, b = 2$ and $f(n) \leq n^{\log_b a - \varepsilon} = n^{\log_2 5 - 0.322} = n^2$ for $\varepsilon = 0.322$. Therefore $T(n) = \theta\big(n^{\log_2 5}\big)$.

S Panbir Singh - Lecture note